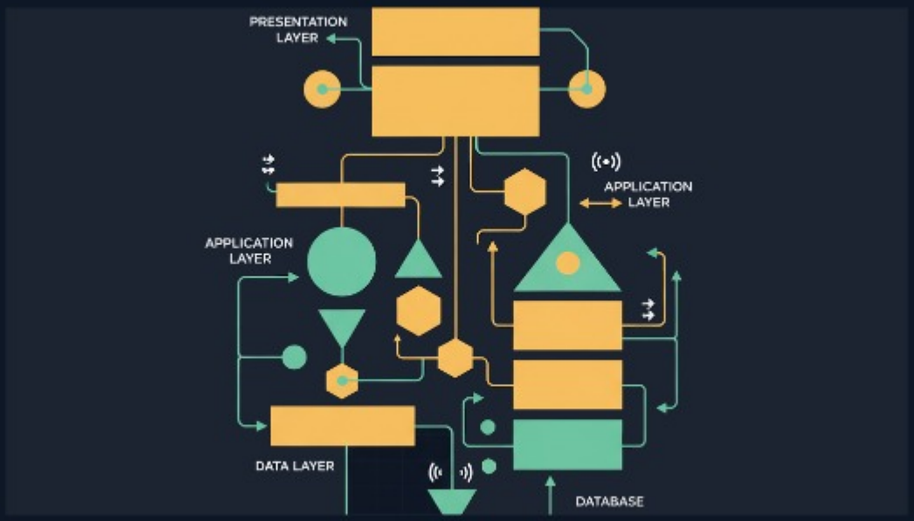
Layered Architecture

Data Access Layer



Acceso a datos en arquitecturas modernas

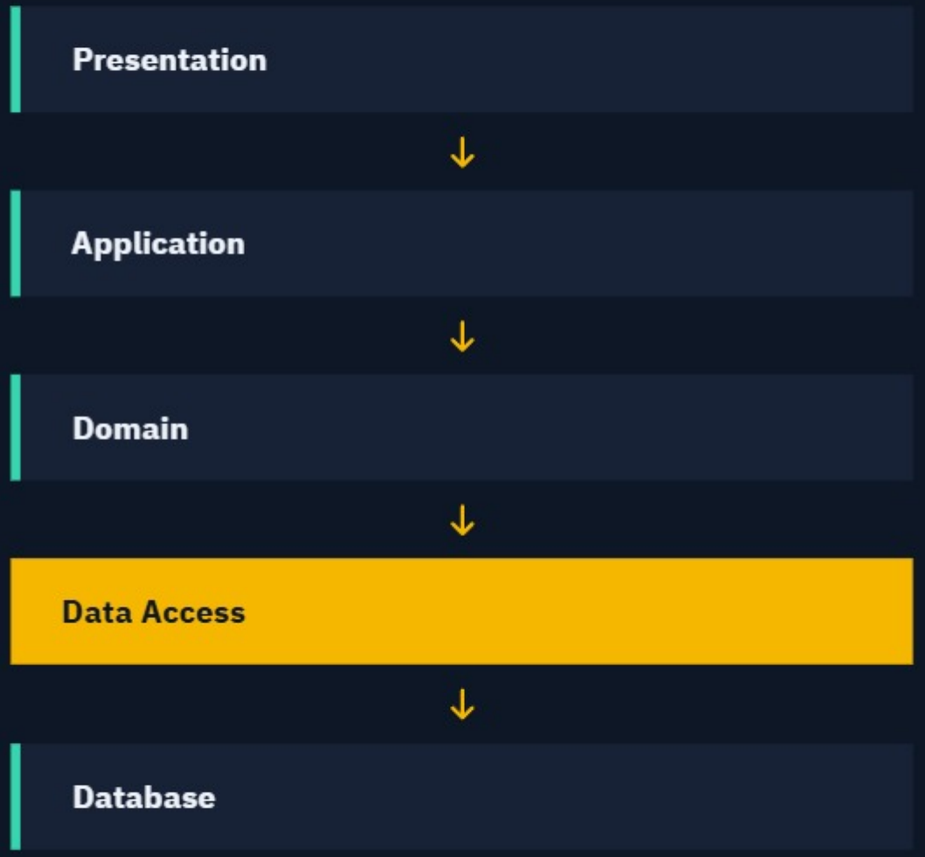
- Instructor: Juan Carlos De La Cruz

¿Qué es Layered Architecture?

Arquitectura de software que organiza el código en capas con responsabilidades específicas, donde cada capa solo se comunica con las adyacentes.

Resuelve: Código difícil de mantener, alto acoplamiento, difícil de probar

- Separación de responsabilidades
- Escalabilidad
- Mantenibilidad
- Testabilidad



¿Qué es el Data Access Layer?

Rol en la arquitectura

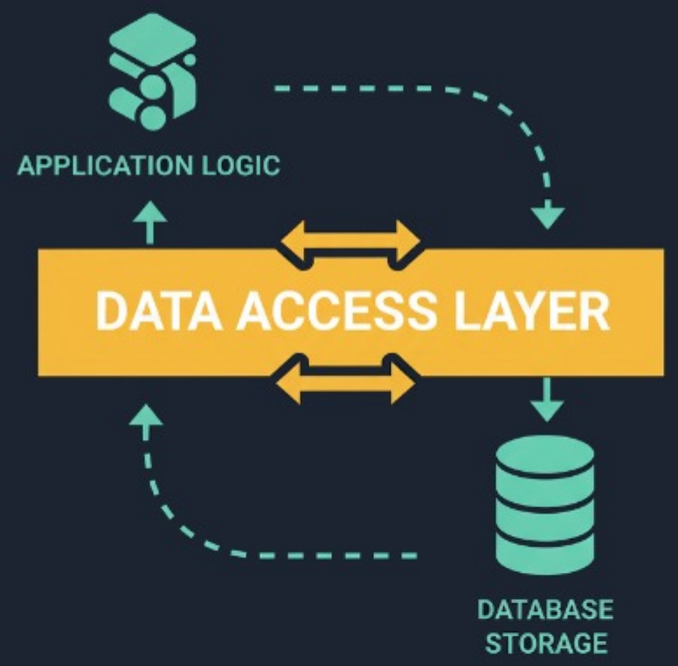
Capa responsable de toda la comunicación entre la aplicación y las fuentes de datos (bases de datos, APIs externas, archivos).

Responsabilidad principal

Abstraer completamente los detalles de persistencia. El resto de la aplicación no debe conocer cómo se almacenan los datos.

Abstracción de datos

Permite cambiar de base de datos (SQL Server → PostgreSQL) sin afectar las capas superiores. Solo se modifica el Data Access Layer.



Responsabilidades del Data Access Layer

Comunicación con la base de datos

Gestiona todas las conexiones y comunicación directa con el motor de base de datos.

Encapsulación de queries

Oculta los queries SQL o llamadas ORM detrás de métodos claros y reutilizables.

CRUD operations

Implementa operaciones básicas: Create, Read, Update, Delete sobre las entidades.

Manejo de conexiones

Controla apertura, cierre y pooling de conexiones de forma eficiente.

Mapeo de entidades

Convierte entre objetos de dominio y estructuras de la base de datos.

Persistencia de datos

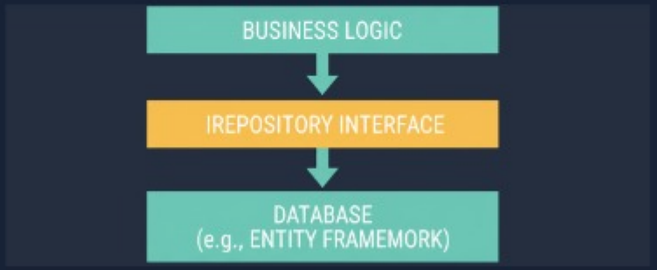
Garantiza que los cambios se guarden correctamente y de forma consistente.

Patrones comunes en Data Access

Repository Pattern

Abstrae el acceso a datos mediante interfaces. Evita que las capas superiores dependan directamente del ORM o base de datos.

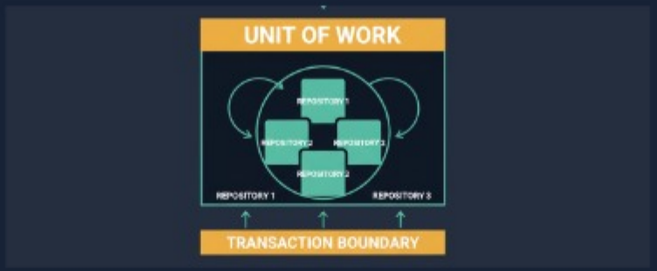
`IRepository → GetById(), GetAll(), Add(), Update(), Delete()`



Unit of Work

Coordina múltiples repositorios en una sola transacción. Garantiza que todos los cambios se confirmen o reviertan juntos.

`UnitOfWork → BeginTransaction(), Commit(), Rollback()`



ORM (Object-Relational Mapping)

Mapea automáticamente objetos del código a tablas de la base de datos. Ejemplo: Entity Framework Core en .NET.

`DbContext → DbSet, SaveChanges(), LINQ queries`

Ejemplo conceptual en .NET

Entidad

```
Product { Id { get; set; } Name { get; set; } Price {
get; set; } }
```

Servicio

```
ProductService { IProductRepository _repo; Product
GetProduct( id) { _repo.GetById(id); } }
```

Repositorio

```
IProductRepository { Product GetById( id);
IEnumerable<Product> GetAll(); Add(Product product); }
```

DbContext

```
AppDbContext : DbContext { DbSet<Product> Products { get;
set; } }
```

Flujo de operación

→ Controller recibe request

→ Service ejecuta lógica

→ Repository consulta DB

← Resultado vuelve

Flujo de una operación

1

Controller recibe request

El usuario hace una petición HTTP GET a /api/products

2

Service ejecuta lógica

ProductService aplica reglas de negocio (validaciones, filtros, permisos)

3

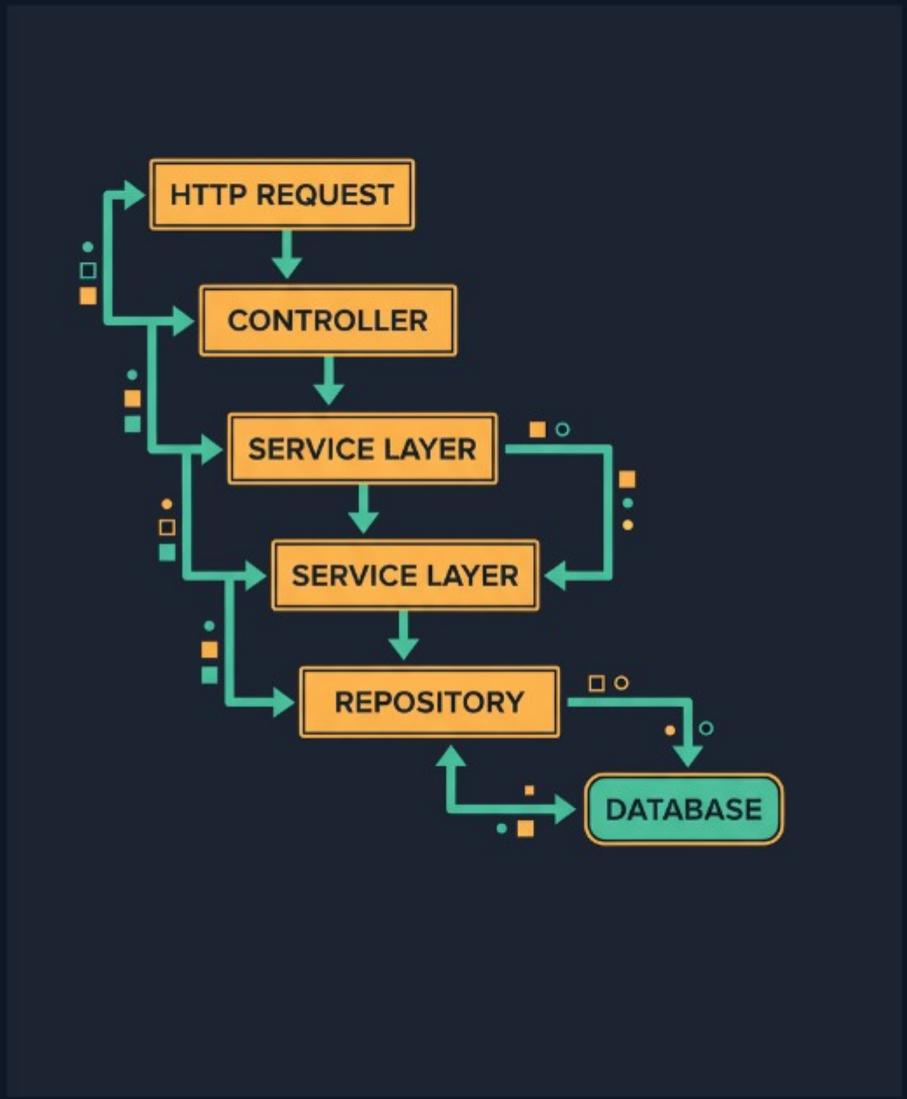
Repository consulta base de datos

ProductRepository ejecuta query contra la base de datos y mapea resultados

4

Resultado vuelve al Controller

Los datos fluyen de vuelta y se serializan como JSON para la respuesta HTTP



Buenas prácticas

No mezclar lógica de negocio en repositorios

Los repositorios solo deben encargarse de persistencia. La lógica de negocio va en los servicios de la capa de aplicación.

Usar interfaces para abstracción

Define contratos con interfaces (IProductRepository) para desacoplar y facilitar testing con mocks.

Implementar inyección de dependencias

Registra repositorios en el contenedor IoC. Evita crear instancias manualmente con new.

Mantener repositorios enfocados en persistencia

Operaciones CRUD y queries específicas. Sin validaciones, cálculos o transformaciones complejas.

Evitar queries en controllers

Los controllers solo deben orquestar. Delega toda interacción con datos a servicios y repositorios.

Anti-patrones comunes

Errores frecuentes que debes evitar

Controllers accediendo directamente a la BD

Rompe la separación de responsabilidades. El controller no debe conocer detalles de persistencia.

`X DbContext directamente en Controller`

SQL embebido en servicios

Los queries deben estar en la capa de datos. Mezclar SQL en servicios hace el código difícil de mantener.

`X Raw SQL en Service Layer`

Repositorios con lógica de negocio

Validaciones, cálculos y reglas de negocio no pertenecen al repositorio. Solo persistencia.

`X Validaciones en Repository`

Capas fuertemente acopladas

Referencias circulares o dependencias directas entre capas hacen el sistema rígido e imposible de probar.

`X Sin interfaces, acoplamiento directo`

Beneficios en proyectos reales

Escalabilidad

Facilita agregar nuevas funcionalidades sin tocar código existente. Puedes cambiar de base de datos o agregar cachés sin modificar servicios.

Tests unitarios más fáciles

Mockear repositorios es simple. Puedes probar servicios sin necesidad de una base de datos real, acelerando el desarrollo.

Código mantenible

Cada capa tiene una responsabilidad clara. Los desarrolladores saben dónde encontrar y modificar código relacionado con datos.

Evolución del sistema

Migrar tecnologías o refactorizar es menos riesgoso. El impacto de cambios está contenido en capas específicas.

Conclusión

El Data Access Layer protege tu arquitectura

Actúa como una barrera que aísla las decisiones de persistencia del resto del sistema, permitiendo que tu aplicación evolucione sin dolor.

Desacopla la persistencia del resto de capas

Es clave para arquitecturas escalables

Facilita mantenimiento y testing

¿Preguntas?